

Dynamic compilation [in Java]

Objectives:

- Understand dynamic compilation process, benefits and limitations
- Identification of performance constraints of the Java language

Introduction

There are many modern programming languages that use dynamic compilation to support multiple platforms. Java is a language that uses dynamic compilation. The process of compiling Java code starts by transforming the code into an intermediate language called bytecode. The bytecode is cross-platform compatible, the machine only needs a Java Virtual Machine to run the code. The Virtual Machine allows the execution of the program in two different ways: interpretation or compilation. The code execution starts with interpretation (of the program bytecodes). Code compilation is performed during the program execution (dynamic compilation or Just-In-Time compilation, JIT) for chunks of code with high computational costs. In many cases, chunks of code are recompiled to apply more complex optimisations. If the optimisations are no longer valid, previously compiled versions will be used. In this lab session, students will analyse the Java compilation process.

For the lab session, the students will need a Java implementation of the matrix multiplication. This and other files required for this session are available on e-learning.

Java Profiling

- Dynamic compilation process: Use `-XX:+PrintCompilation` in program execution to identify the methods which were compiled to native instructions. How many times was `mmult` compiled? Why?
- The `perfreport1.txt` file contains the execution profile of the same run. Identify the main program hotspots. Justify, why the `mmult` method appears a different number of times from a).
- Identify code executed: The file `perfreport1asm.txt` contains the assembly code generated by the JIT during program execution. Identify the assembly code in the hotspot. (*) **Extra:** identify this loop in the bytecode (use `javap -c MULT.class` to see the bytecode)
- Analyse code: The Java program requires more instructions than C program to perform the matrix multiplication. Analyse assembly code generated from Java (kernel of matrix multiplication) and identify optimisations present in the code. These optimisations were also applied in the C version?
- Compiler impact: Open the file `perfreport2.txt` and analyse if there is a negative impact of dynamic compilation on performance. Do you see any other important source of overhead on this profile?
- Garbage Collector: The Java Virtual Machine uses a Garbage Collector to manage memory at runtime. The Garbage Collector sometimes needs to interrupt the program execution to reclaim unused program storage. Identify if the cost of the Garbage Collector is relevant? Compare the file `perfreport2.txt` (run of the `MULT` class) and `perfreport3.txt` (run of the `MULT_2` class). Why?

- g) (*) Measure and compare Java with C: Fill the table with performance data collected with `perf stat -M cpi ./a.out`. Calculate the performance gain between the C and Java versions and comment the results.

N	Version	Time	CPI	#I (inst_retired.any)	L1_DMiss	Miss/#I
512	C					
	Java					

Commands to use:

Load Java sdk (currently v21.0.8)

```
ml Java
```

Compile a Java program:

```
javac MULT.java
```

Run program (java bytecode):

```
java MULT
```

Generate perfreport1.txt

```
perf record -k 1 -F 1000 java -XX:+UnlockDiagnosticVMOptions -XX:+PreserveFramePointer -XX:+DebugNonSafepoints -agentpath:/usr/lib64/libperf-jvmti.so MULT
```

```
perf inject -i perf.data --jit -o perf.data.jitted
```

```
perf report -i perf.data.jitted --stdio > perfreport1.txt
```

Generate perfreport1asm.txt

```
perf annotate -i perf.data.jitted --stdio > perfreport1asm.txt
```